

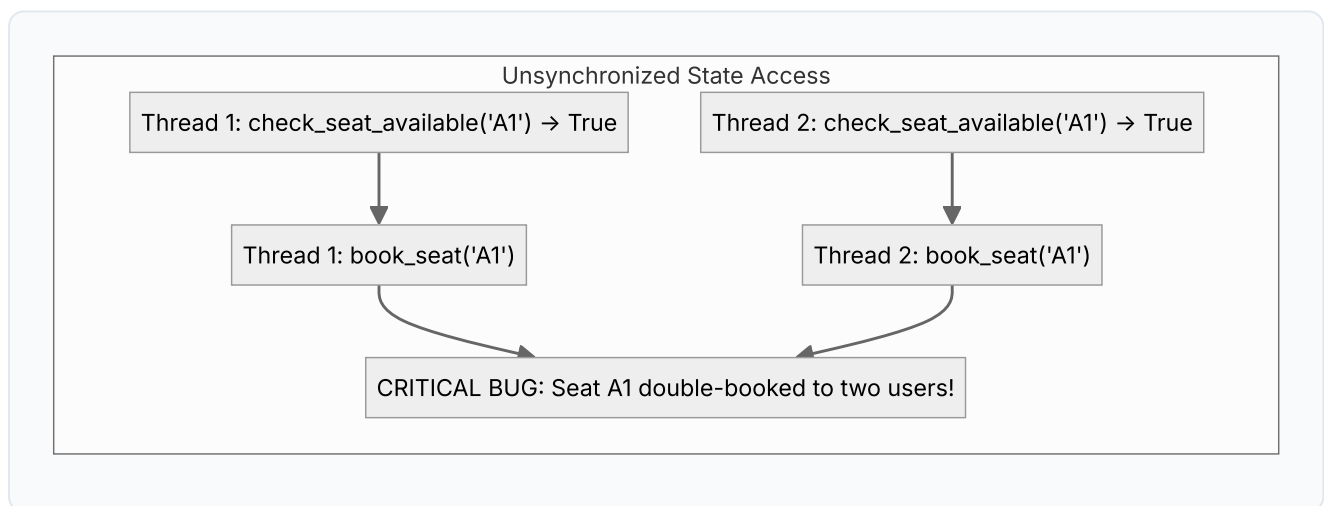
LLD Chapter 4: Concurrency Patterns, Thread-Safety & Locking in System Design

Core Learning Objective: Master concurrency management in Low-Level Design interviews. Learn how to write thread-safe systems, implement a Readers-Writer lock from scratch, prevent deadlocks, and use double-checked locking in Python.

1. Why Concurrency Matters in LLD Interviews

In real-world Product-Based Company interviews, **anyone can write an object-oriented class diagram**. The true differentiator between a junior engineer and a Senior/Staff Engineer is **demonstrating thread-safety and concurrency controls**:

- What happens when 1,000 concurrent threads attempt to reserve the same movie seat simultaneously?
- How do you prevent double-charging or race conditions in an inventory system?
- How do you allow high-speed concurrent reads without blocking while ensuring atomic writes?



2. The Readers-Writer Lock (RWLock) Pattern

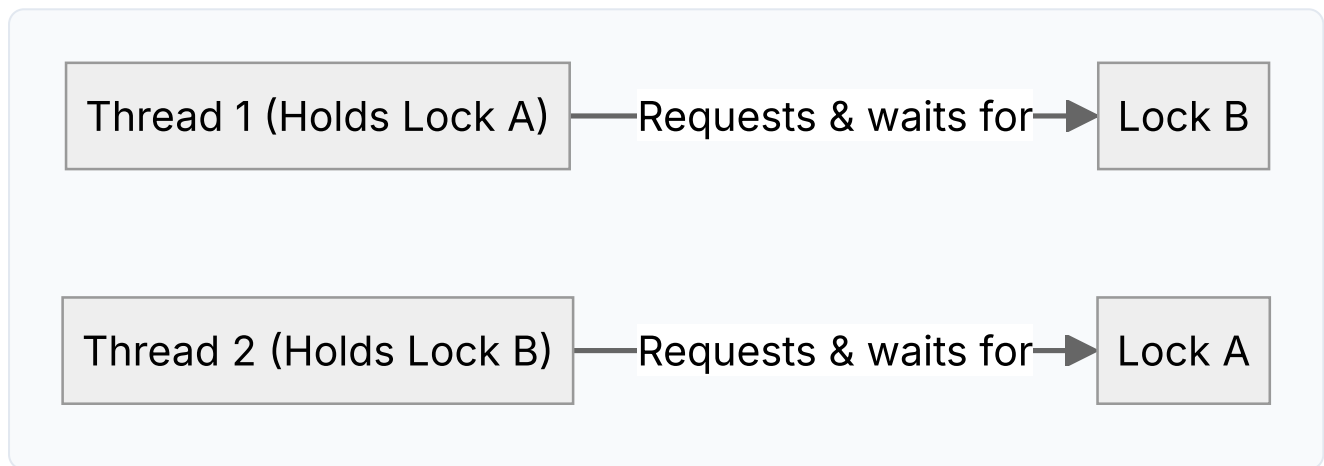
In systems like Bookings, Catalogs, or Config stores, 99% of operations are **Reads**, and only 1% are **Writes**. Using a simple `threading.Lock` forces all readers to wait in a single-file line, destroying throughput. A **Readers-Writer Lock (Shared-Exclusive Lock)** allows:

- **Multiple concurrent Readers** simultaneously.
- **Only ONE Writer** at a time (mutually exclusive with all readers and writers).

Complete Thread-Safe RWLock Implementation in Python

3. Deadlocks & The 4 Coffman Conditions

A **deadlock** is a state where a set of threads are permanently blocked because each holds a lock the other requires.



The 4 Coffman Conditions for Deadlock:

1. **Mutual Exclusion:** Resources cannot be shared; locks must be exclusive.
2. **Hold and Wait:** A thread holds at least one lock while waiting to acquire another.
3. **No Preemption:** Locks cannot be forcibly confiscated from a thread.
4. **Circular Wait:** Thread 1 waits on Thread 2, which waits on Thread 1.

How to Eliminate Deadlocks: Global Lock Ordering

The most reliable way to break the Circular Wait condition is **Global Strict Lock Ordering**: Whenever a thread needs to acquire multiple locks, it **must always acquire them in deterministic alphabetical or ID order**:

Regardless of whether Thread 1 transfers $A \rightarrow B$ and Thread 2 transfers $B \rightarrow A$, both threads attempt to acquire `min(A.id, B.id)` first! Circular wait is mathematically impossible.

4. Double-Checked Locking in Python

Used to initialize expensive singleton resources without paying synchronization lock overhead on every subsequent read access:



5. Check-Then-Act: The Race You Will Be Asked to Fix

The most common concurrency bug in a design interview is a check followed by an action with a gap between them. A barrier forces the bad interleaving every time, so the bug is reproducible:



The same shape appears as "check seat available, then book", "check stock, then deduct" and "if key not in cache, then compute and store". The fix is always one of three moves: hold a lock across both steps, use one atomic operation (`dict.setdefault`, `queue.Queue.put`, a database `UPDATE ... WHERE balance ≥ :amount`), or make the operation idempotent.

6. Producer-Consumer with a Bounded Queue



Points to say aloud: the queue does its own locking, the bound prevents unbounded memory growth, and **one sentinel per consumer** is the clean shutdown. This is the skeleton behind the logging framework, the job scheduler and the message broker case studies.

7. Choosing a Concurrency Tool

Workload	Tool	Why
Many waiting I/O calls (HTTP, DB)	<code>asyncio</code> or a thread pool	Waiting releases the GIL or yields; the CPU is idle anyway
CPU-bound pure Python	<code>multiprocessing</code> or <code>ProcessPoolExecutor</code>	The GIL prevents threads from running Python bytecode in parallel (on free-threaded builds this changes)
Shared mutable state, few writers	<code>threading.Lock</code> or <code>RLock</code>	Smallest correct tool
Many readers, rare writers	A readers-writer lock (section 2)	Readers do not block each other
Hand work between threads	<code>queue.Queue</code>	Built-in locking and backpressure
Wait for an event once	<code>threading.Event</code>	No polling
Limit concurrent access to a resource	<code>threading.Semaphore</code>	Counted permits (connection pool)

Lock hygiene checklist

1. Hold locks for as short a time as possible and never across network or disk I/O.
2. Acquire multiple locks in one global order (the sorted seat ids in the booking system).
3. Prefer `with lock:` so an exception cannot leave a lock held.
4. Never call unknown code (callbacks, observers) while holding a lock: it can re-enter and deadlock, as the job scheduler case study shows.
5. Make shared state immutable where you can; an immutable object needs no lock.

Further Reading

- [threading module](#)
- [queue module](#)
- [Wikipedia: Deadlock](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Why is `count += 1` not thread-safe?

It is read-modify-write across several bytecodes; two threads can read the same value and one update is lost. Use a lock or atomic structure.

▼ 2. What are the four conditions for deadlock?

Mutual exclusion, hold-and-wait, no preemption, circular wait. Break one (usually enforce a global lock order).

▼ 3. Why prefer a concurrent queue between producers and consumers?

It decouples rates, gives backpressure (bounded queue) and keeps shared-state locking in one tested place.

▼ 4. When use a read-write lock?

When reads vastly outnumber writes and readers can proceed concurrently.